

Mini Project Report:

Tic-Tac-Toe Game

จัดทำโดย

- 1) นายก้องกิตากร เตชรัตน์ 68010052
- 2) นางสาวเกตุแก้ว ชมกลิ่น 68010104
- 3) นายศักรินทร์ คล้ายคลึง 68011056
- 4) นางสาวตุลยา ยอดแก้ว 68011697

Concept

หลักการทำงานของ Tic-Tac-Toe Game

เกม OX (Tic-Tac-Toe) ถูกพัฒนาให้เล่นบน TFT LCD ขนาด 1.8 นิ้ว ใช้ PS2 XY Joystick สำหรับการควบคุมการเลือกตำแหน่งและกดปุ่มบน Joystick เพื่อวางสัญลักษณ์ X มีระบบตรวจสอบผู้ชนะและแสดงผลหลังจากเกมจบ จะเป็นการเล่นกับบอทที่ถูกออกแบบมาเพื่อแข่งขันกับผู้เล่น โดยจะเป็นการสุ่มเลือกผู้เริ่มก่อน เช่น หากบอทได้เริ่มก่อนเมื่อจบตาของบอทก็จะมีโอกาสชนะเกิดขึ้นหรือไม่หากไม่มีจะไปที่ตาของผู้เล่น และเช็คการชนะสลับกันจนกว่าจะมีการชนะเพื่อเป็นการจบเกม หลังจากจบเกมจะมีการรีเซ็ตเพื่อเคลียร์ตารางแล้วสุ่มผู้เริ่มก่อนอีกครั้ง

การทำงานของบอท

บอทจะถูกออกแบบมาเพื่อหาตำแหน่งที่เหมาะสมที่สุดในการวาง O เพื่อเอาชนะเกมนี้ และเมื่อผู้เล่นมีการวาง X ที่ตำแหน่งที่เลือกบอทก็จะมีหน้าที่ในการป้องกันตำแหน่งที่มีโอกาสที่จะทำให้ผู้เล่นชนะโดยการวาง O ลงไปในตำแหน่งนั้น โดยการป้องกันตำแหน่งที่จะทำให้ผู้เล่นชนะจะต้องคำนึงถึงโอกาสที่บอทจะชนะเช่นกัน

การชนะเกม OX (Tic-Tac-Toe)

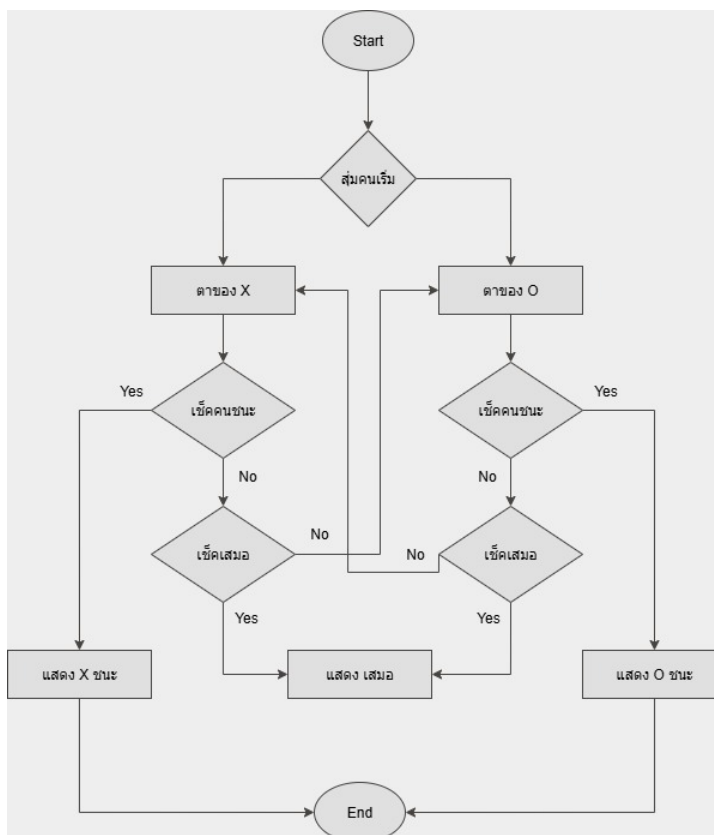
รูปแบบการชนะมีทั้งหมด 8 รูปแบบดังนี้

แบบที่ 1	แบบที่ 2	แบบที่ 3	แบบที่ 4
0 1 2	0 1 2	0 1 2	0 1 2
3 4 5	3 4 5	3 4 5	3 4 5
6 7 8	6 7 8	6 7 8	6 7 8

แบบที่ 5	แบบที่ 6	แบบที่ 7	แบบที่ 8
0 1 2	0 1 2	0 1 2	0 1 2
3 4 5	3 4 5	3 4 5	3 4 5
6 7 8	6 7 8	6 7 8	6 7 8

Flowchart

หลักการทำงานของเกม OX จะเป็นดังนี้



Software Design

โครงสร้าง Software ของเกม

ประกอบด้วย 3 ส่วน ได้แก่

- mainOX.ino → ไฟล์หลักในการจัดเก็บข้อมูลตาราง, ตำแหน่ง และการตรวจสอบการชนะ
- OXgame.ino → การวิเคราะห์การเล่นของบอท, การควบคุมสถานะ (State Machine) และลำดับในการเล่น
- stdfunc.ino → ฟังก์ชันมาตรฐาน ได้แก่ การวาดตาราง, การเลื่อนสัญลักษณ์ X และการแสดงผู้ชนะ

mainOX

- กำหนดตารางเริ่มต้น, เคลียร์ตารางให้โล่ง
- เก็บค่าตำแหน่ง พิกัด X, Y ที่สามารถลงได้
- กำหนด รูปแบบการชนะ เช่น แนวตั้ง แนวนอน แนวแยงใน possible_line_component
- มี struct สำหรับเก็บข้อมูลการเล่นแต่ละตำแหน่ง
- สำหรับเก็บข้อมูลบอท เพื่อใช้ในการเล่น

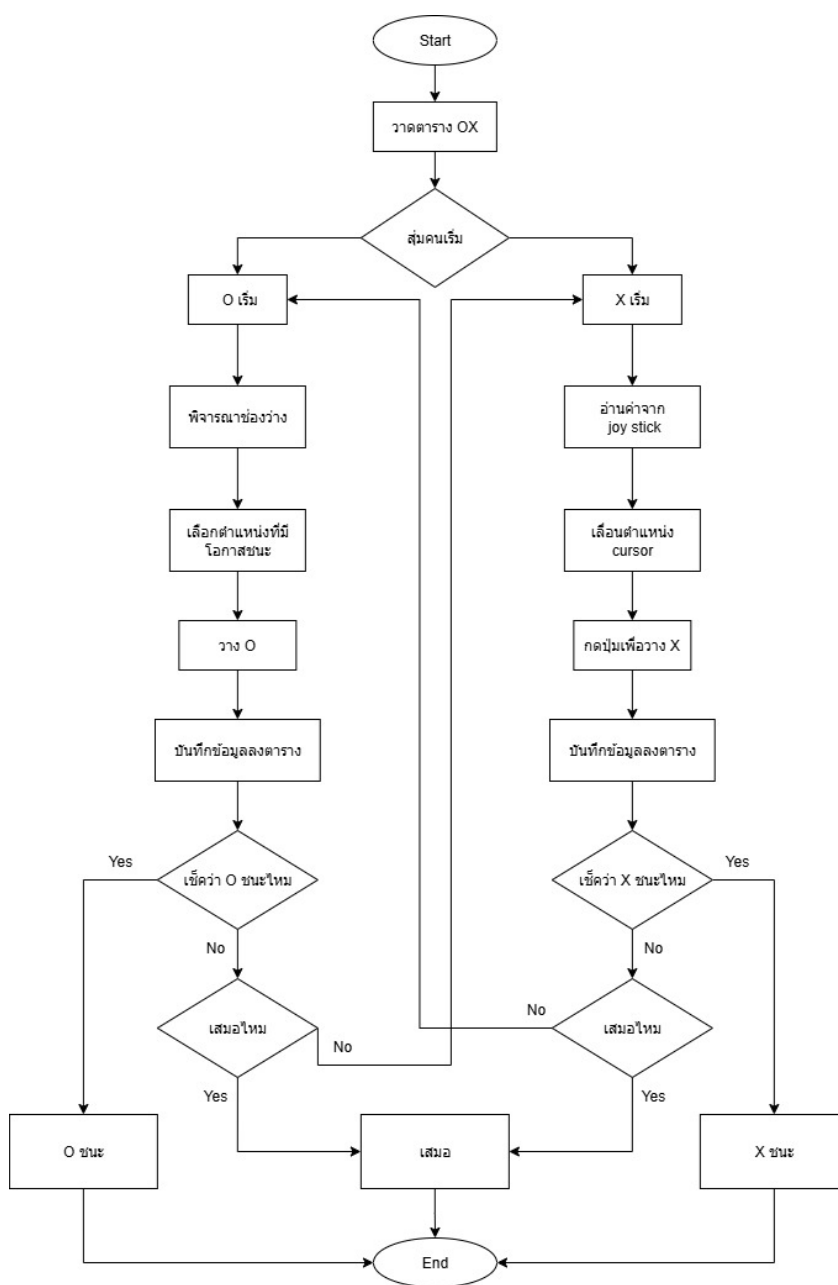
OXgame

- ใช้ Finite State Machine (FSM) ในการจัดการการเล่น
 - Start → เริ่มต้นเกม, เคลียร์บอร์ด
 - Xturn → ผู้เล่น X เลือกตำแหน่ง
 - Oturn → ผู้เล่น O เลือกตำแหน่ง
 - CheckWin → ตรวจสอบผลว่ามีผู้ชนะหรือไม่
- ใช้ Joystick Input (VRx, VRy, SW) เพื่อตรวจจับทิศทางการเลื่อนและการเลือก
- เมื่อเกมจบ → เปลี่ยนไปยังสถานการณ์แสดงผล

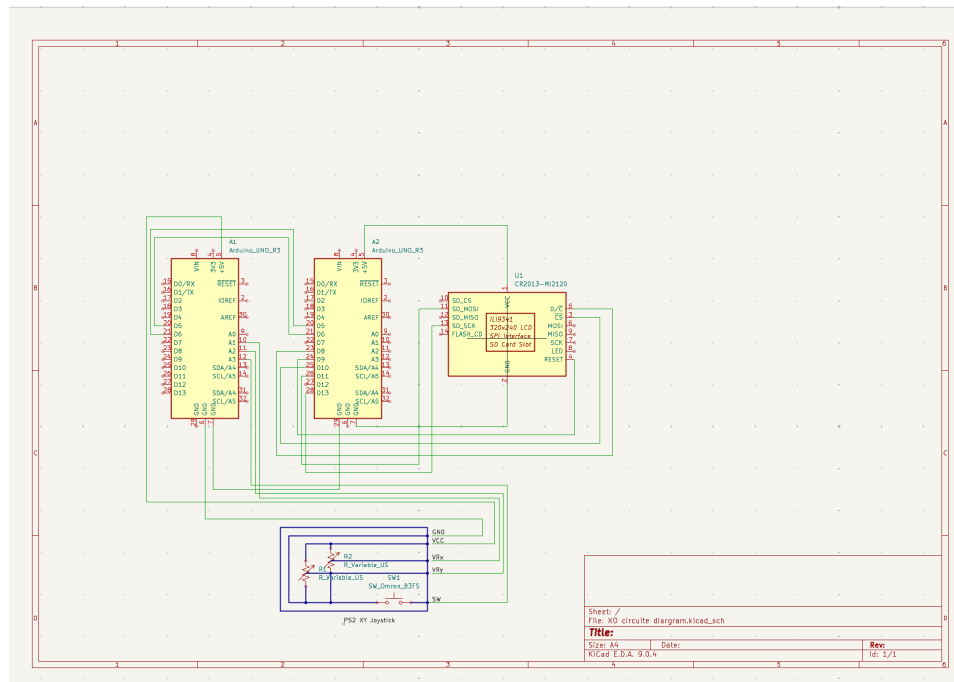
stdfunc

- draw_grid → สร้างตาราง 3x3 บนหน้าจอ TFT LCD
- ShowWinner or Draw → แสดงผลผู้ชนะ (O หรือ X) บนหน้าจอ หรือ แสดงผลเสมอ
- แสดงการเลื่อน X ของผู้เล่น และแสดงผลการวาง X บนช่องที่เลือก

Flowchart การทำงานของ Software



Circuit Diagram

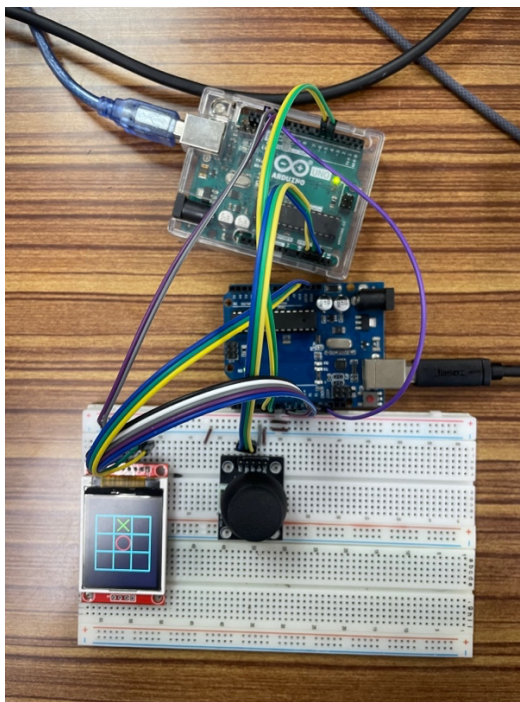


Hardware

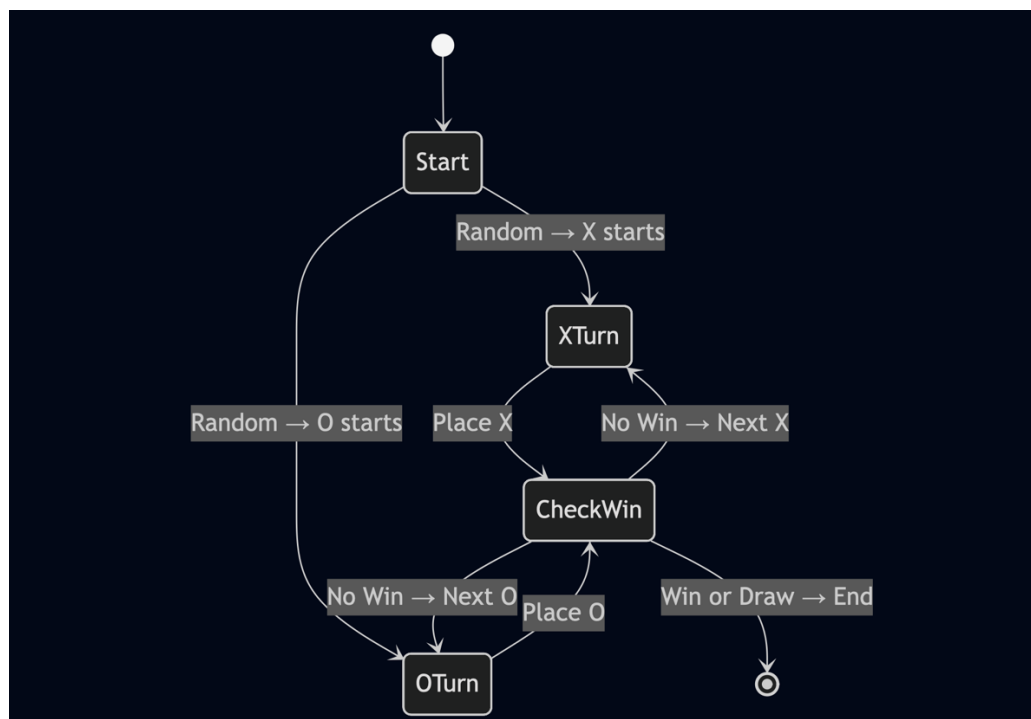
อุปกรณ์ ที่ใช้ในการทำ Mini Project ได้แก่

- 1) Arduino (2)
- 2) Breadboard (2)
- 3) PS2 XY Joystick
- 4) จอ TFT LCD ขนาด 1.8 นิ้ว
- 5) สายไฟ จัมเปอร์

การต่อโมดูล



State diagram



Source Code

OXgame

```
#include <Adafruit_GFX.h>
#include <Adafruit_ST7735.h>
#include <SPI.h>

#define TFT_CS 10
#define TFT_RST 8
#define TFT_DC 9

#define VRx 0
#define VRy 1
#define SW 2

#define Start 0
#define Xturn 1
#define Oturn 2
#define CheckWin 3

#define IP 5
#define OP 6

#include <SoftwareSerial.h>
SoftwareSerial mySerial(5, 6);

long player;
int State = Start;
int PreState = 0;

Adafruit_ST7735 tft = Adafruit_ST7735(TFT_CS, TFT_DC, TFT_RST);

struct str {
    unsigned long Time;
    unsigned long NextSt[3];
};

struct str FSM[4] = {
    { 100, { Xturn, Oturn } },
    { 100, { CheckWin } },
    { 100, { CheckWin } },
    { 2000, { Oturn, Xturn } }
};
```

```

int SpaceIndexCount = -1;

void draw_grid();
void draw_pin();
void playerTempPin();
void bot();
int CheckWinFunc(int target);
void ShowWinner(int winner);
void Draw();

void setup() {
    randomSeed(analogRead(A4));
    Serial.begin(9600);
    mySerial.begin(9600);
    Serial.begin(9600);
    pinMode(7, OUTPUT);
    digitalWrite(7, 1);
    tft.initR(INITR_BLACKTAB);
    tft.setRotation(2);
    tft.fillScreen(ST77XX_BLACK);
}

void loop() {
    delay(300);
    switch (State) {
        case Start:
        {
            HARDmemoryReset();
            draw_grid();
            draw_pin();
            player = random(1, 3);
            Serial.println(player);
            delay(FSM[State].Time);
            if (player == 1) {
                State = Oturn;
            } else {
                State = Xturn;
            }
        }
        break;

        case Xturn:
        {
            Serial.println("X ໓໓໓໓໓");
            playerTempPin();
            delay(FSM[State].Time);

```



```

        PreState = Xturn;
        State = CheckWin;
    }
    break;

case Oturn:
{
    Serial.println("เข้า 0 แล้ว");
    bot();
    delay(FSM[State].Time);
    PreState = Oturn;
    State = CheckWin;
}
break;

case CheckWin:
{
    int X_win = CheckWinFunc(2);
    int O_win = CheckWinFunc(1);
    if (X_win >= 0) ShowWinner(2);
    if (O_win >= 0) ShowWinner(1);
    if (how_many_space() == 0 && !(X_win >= 0 || O_win >= 0)) Draw();
    if (!(X_win >= 0 || O_win >= 0) && how_many_space() != 0) {
        State = 3 - PreState;
        return;
    } else State = Start;
    delay(FSM[CheckWin].Time);
}
break;
}
}

```

mainOX

```
int pins[9] = { 0, 0, 0, 0, 0, 0, 0, 0, 0 };

int pins_Xposition[9] = { 31, 64, 97, 31, 64, 97, 31, 64, 97 };

int pins_Yposition[9] = { 47, 47, 47, 80, 80, 80, 113, 113, 113 };

int possible_line_component[8][3] = { { 0, 3, 6 }, { 1, 4, 7 }, { 2, 5, 8 }, { 0, 1, 2 }, { 3, 4, 5 }, { 6, 7, 8 }, { 0, 4, 8 }, { 2, 4, 6 } };

struct dataset {
    int check;
    int position;
    int possible_line;
    int possible_lineIndex[8];
    int possible_member[8];
};

struct dataset pin0[10];
struct dataset pinX[10];

void possible_lineANDmemberCheck(int pin) {
    for (int target = 1; target <= 2; target++) {
        int target1 = 3 - target;
        for (int i = 0; i < 8; i++) {
            if (((pins[possible_line_component[i][0]] == target) ||
(pins[possible_line_component[i][1]] == target) ||
(pins[possible_line_component[i][2]] == target)) &&
!((pins[possible_line_component[i][0]] == target1) ||
(pins[possible_line_component[i][1]] == target1) ||
(pins[possible_line_component[i][2]] == target1))) {
                if (target == 1) {
                    pin0[pin].check = 1;
                    pin0[pin].possible_lineIndex[i] = 1;
                    pin0[pin].possible_line++;
                } else if (target == 2) {
                    pinX[pin].check = 1;
                    pinX[pin].possible_lineIndex[i] = 1;
                    pinX[pin].possible_line++;
                }
            }
        }
    }
}

for (int i = 0; i < 8; i++) {
    if (pin0[pin].possible_lineIndex[i] == 1) {
        for (int j = 0; j < 3; j++) {
```

```

        if (pins[possible_line_component[i][j]] == 1) {
            pin0[pin].possible_member[i]++;
        }
    }
}
if (pinX[pin].possible_lineIndex[i] == 1) {
    for (int j = 0; j < 3; j++) {
        if (pins[possible_line_component[i][j]] == 2) {
            pinX[pin].possible_member[i]++;
        }
    }
}
}
}

int space[9] = { 0 };
int how_many_space() {
    memset(space, 0, sizeof(space));
    int spaceIndex = 0;
    for (int i = 0; i < 9; i++) {
        if (pins[i] == 0) {
            space[spaceIndex++] = i;
        }
    }
    return spaceIndex;
}

int tempPin;
void advance_to_next_empty_pin() {
    int numSpace = how_many_space();
    static int last = -1;
    int current = -1;
    if (last >= 0 && last < 9 && pins[last] == 1) {
        current = last;
    }
    int pick = -1;
    for (int k = 0; k < numSpace; k++) {
        if (space[k] > current) {
            pick = space[k];
            break;
        }
    }
    if (pick == -1) pick = space[0];
    if (current >= 0 && current < 9 && pins[current] == 1) {
        pins[current] = 0;
    }
    pins[pick] = 1;
}

```

```

    last = pick;
    tempPin = pick;
}

void bot() {
    memset(pin0, 0, sizeof(pin0));
    memset(pinX, 0, sizeof(pinX));
    for (int pin = 0; pin < 10; pin++) {
        pin0[pin].possible_line = 0;
        pinX[pin].possible_line = 0;
        memset(pin0[pin].possible_member, 0, sizeof(pin0[pin].possible_member));
        memset(pinX[pin].possible_member, 0, sizeof(pinX[pin].possible_member));
        memset(pin0[pin].possible_lineIndex, 0, sizeof(pin0[pin].possible_lineIndex));
        memset(pinX[pin].possible_lineIndex, 0, sizeof(pinX[pin].possible_lineIndex));
    }
    possible_lineANDmemberCheck(9);
    how_many_space();
    int space0 = how_many_space();
    if (space0 == 9) {
        int preferred[] = { 0, 2, 4, 6, 8 };
        int r = random(0, 5);
        pins[preferred[r]] = 1;
        draw_pin();
        return;
    }

    for (int event = 0; event < space0; event++) {
        advance_to_next_empty_pin();
        possible_lineANDmemberCheck(tempPin);
        pin0[tempPin].position = tempPin;
        pin0[tempPin].check = 1;
    }
    pins[tempPin] = 0;

    struct target_dataset {
        int position;
        int line_mine_gain;
        int member_mine_gain;
        int special_member_mine_gain;
        int line_enemy_loss;
        int member_enemy_loss;
        int special_member_enemy_loss;
        int score;
    };
    struct target_dataset targetPin[space0];
    memset(targetPin, 0, sizeof(targetPin));

```

```

int targetPinIndex = 0;
for (int i = 0; i < 9; i++) {
    if (pin0[i].check) {
        targetPin[targetPinIndex].position = pin0[i].position;
        targetPin[targetPinIndex].line_mine_gain = pin0[i].possible_line -
pin0[9].possible_line;
        targetPin[targetPinIndex].line_enemy_loss = pinX[9].possible_line -
pinX[i].possible_line;
        targetPin[targetPinIndex].member_mine_gain = 0;
        targetPin[targetPinIndex].special_member_mine_gain = 0;
        targetPin[targetPinIndex].member_enemy_loss = 0;
        targetPin[targetPinIndex].special_member_enemy_loss = 0;
        for (int j = 0; j < 8; j++) {
            targetPin[targetPinIndex].member_mine_gain += pin0[i].possible_member[j];
            targetPin[targetPinIndex].member_mine_gain -= pin0[9].possible_member[j];
            targetPin[targetPinIndex].member_enemy_loss += pinX[9].possible_member[j];
            targetPin[targetPinIndex].member_enemy_loss -= pinX[i].possible_member[j];
        }
        targetPinIndex++;
    }
}
int maxscore = -10;
int maxscorePin = 1;
for (int i = 0; i < targetPinIndex; i++) {
    targetPin[i].score = targetPin[i].line_mine_gain * 10 +
targetPin[i].member_mine_gain * 10 + targetPin[i].line_enemy_loss * 10 +
targetPin[i].member_enemy_loss * 10;

    if (targetPin[i].score > maxscore) {
        maxscore = targetPin[i].score;
        maxscorePin = targetPin[i].position;
    }
}
pins[maxscorePin] = 1;
draw_pin();
}

```

Stdfunc

```

void draw_grid() {
    tft.initR(INITR_BLACKTAB);
    tft.setRotation(2);
    tft.fillScreen(ST77XX_BLACK);
    uint16_t color = ST77XX_CYAN;
    tft.drawLine(14, 30, 14, 130, color);
    tft.drawLine(47, 30, 47, 130, color);
    tft.drawLine(80, 30, 80, 130, color);
    tft.drawLine(114, 30, 114, 130, color);
    tft.drawLine(14, 30, 114, 30, color);
    tft.drawLine(14, 63, 114, 63, color);
    tft.drawLine(14, 96, 114, 96, color);
    tft.drawLine(14, 130, 114, 130, color);
}

```

```

void ShowWinner(int winner) {
    tft.initR(INITR_BLACKTAB);
    tft.setRotation(2);
    tft.fillScreen(ST77XX_BLACK);
    tft.setCursor(22, 50);
    tft.setTextColor(ST77XX_GREEN);
    tft.setTextSize(2);
    tft.println("Winner!");
    tft.setCursor(56, 80);
    tft.setTextColor(ST77XX_GREEN);
    tft.setTextSize(3);
    if (winner == 1) tft.println("O");
    if (winner == 2) tft.println("X");
}

```

```

void Draw() {
    tft.initR(INITR_BLACKTAB);
    tft.setRotation(2);
    tft.fillScreen(ST77XX_BLACK);
    tft.setCursor(40, 50);
    tft.setTextColor(ST77XX_GREEN);
    tft.setTextSize(2);
    tft.println("Draw");
}

```

```

int CheckWinFunc(int target) {
    for (int i = 0; i < 8; i++) {
        for (int j = 0; j < 3; j++) {
            bool check;

```

```

        if (pins[possible_line_component[i][j]] == target) {
            if (j == 2) return i;
        } else break;
    }
}
return -1;
}

int consoleSerial() {
    while (1) {
        if (mySerial.available()) {
            int number = mySerial.parseInt();
            return number;
        }
    }
}

void drawCross(int i) {
    tft.drawLine(pins_Xposition[i] - 10, pins_Yposition[i] - 10, pins_Xposition[i] + 10,
pins_Yposition[i] + 10, ST77XX_GREEN);
    tft.drawLine(pins_Xposition[i] - 10, pins_Yposition[i] + 10, pins_Xposition[i] + 10,
pins_Yposition[i] - 10, ST77XX_GREEN);
}

void eraseCross(int i) {
    tft.drawLine(pins_Xposition[i] - 10, pins_Yposition[i] - 10, pins_Xposition[i] + 10,
pins_Yposition[i] + 10, ST77XX_BLACK);
    tft.drawLine(pins_Xposition[i] - 10, pins_Yposition[i] + 10, pins_Xposition[i] + 10,
pins_Yposition[i] - 10, ST77XX_BLACK);
}

int SpaceIndexlist[9] = { -1 };
int selIdx = -1;
int tempPinPlayer = -1;

void memoryReset() {
    selIdx = -1;
    tempPinPlayer = -1;
    SpaceIndexCount = -1;
}

void HARDmemoryReset() {
    for (int i = 0; i < 9; i++) {
        pins[i] = 0;
    }
}

```

```

void makeSpaceIndexList() {
    memset(SpaceIndexlist, -1, sizeof(SpaceIndexlist));
    int count = 0;
    for (int i = 0; i < 9; i++) {
        if (pins[i] == 0) {
            SpaceIndexlist[count++] = i;
        }
    }
    SpaceIndexCount = count;
}

void playerTempPin() {
    memoryReset();
    if (tempPinPlayer == -1) {
        makeSpaceIndexList();
        selIdx = 0;
        tempPinPlayer = SpaceIndexlist[selIdx];
        drawCross(tempPinPlayer);
    }
    while (1) {
        int action = consoleSerial();

        if (action == 2 && SpaceIndexCount > 0) {
            eraseCross(tempPinPlayer);
            selIdx = (selIdx + 1) % SpaceIndexCount;
            tempPinPlayer = SpaceIndexlist[selIdx];
            drawCross(tempPinPlayer);
            delay(350);
        }

        if (action == 1 && SpaceIndexCount > 0) {
            if (selIdx > 0) {
                eraseCross(tempPinPlayer);
                selIdx = selIdx - 1;
                tempPinPlayer = SpaceIndexlist[selIdx];
                drawCross(tempPinPlayer);
                delay(350);
            }
        }

        if (action == 5) {
            eraseCross(tempPinPlayer);
            pins[tempPinPlayer] = 2;
            draw_pin();
            return;
        }
    }
}

```



```

    }
}

void draw_pin() {
    for (int i = 0; i < 9; i++) {
        if (pins[i] == 1) tft.drawCircle(pins_Xposition[i] + 1, pins_Yposition[i], 13,
ST77XX_RED);
        if (pins[i] == 2) {
            tft.drawLine(pins_Xposition[i] - 10, pins_Yposition[i] - 10, pins_Xposition[i] +
10, pins_Yposition[i] + 10, ST77XX_YELLOW);
            tft.drawLine(pins_Xposition[i] - 10, pins_Yposition[i] + 10, pins_Xposition[i] +
10, pins_Yposition[i] - 10, ST77XX_YELLOW);
        }
    }
}
}

```

OXgame Joystick

ใช้รับ input จาก joystick เพื่อส่งไปยัง Arduino อีกตัว

```

#define VRy 2
#define SW 3

#include <SoftwareSerial.h>
SoftwareSerial mySerial(5, 6);

void setup() {
    Serial.begin(9600);
    mySerial.begin(9600);
    Serial.begin(9600);
    pinMode(12, OUTPUT);
    pinMode(13, OUTPUT);
    digitalWrite(12, 1);
    digitalWrite(13, 0);
}

int console(int VRyc, int SWc) {
    int VRy_value = analogRead(VRyc);
    int SW_value = analogRead(SWc);
    if (VRy_value > 1000) return 1;
    if (VRy_value < 100) return 2;
    if (SW_value < 20) return 5;
    return 0;
}

```

```
int consoleCheck() {
    int VRx_value = analogRead(1);
    int VRy_value = analogRead(2);
    int SW_value = analogRead(3);
    Serial.print("VRx_value => ");
    Serial.println(VRx_value);
    Serial.print("VRy_value => ");
    Serial.println(VRy_value);
    Serial.print("SW_value => ");
    Serial.println(SW_value);
    Serial.println("-----");
}

void loop() {
    mySerial.println(console(VRy, SW));
    delay(200);
}
```